

PROYECTO FINAL · FULLSTACK

ReNodo

La biblioteca de objetos de barrio que convierte compras puntuales en recursos compartidos.

NODE + EXPRESS

API REST segura

REACT

UX accesible

MONGODB

Datos relacionados

CLOUDINARY

Imágenes reales

LEAFLET

Mapa interactivo

MEMORIA TÉCNICA Y DE PRODUCTO

Curso FullStack · Entrega final
septiembre 2026

<https://renodo-web.onrender.com>

01 · SENTIDO DEL PROYECTO

Compartir más. Comprar menos.

ReNodo está pensado para personas urbanas que necesitan objetos de uso ocasional y para centros vecinales que quieren gestionar esos préstamos con trazabilidad.

0 €	12	180
coste del préstamo	centros de barrio	objetos disponibles

Problema que resuelve y propuesta de valor

Compras de un solo uso Taladros, proyectores o tiendas de campaña se compran para usarse muy pocas veces.	Alternativas dispersas La opción de alquiler suele estar dispersa, ser cara o exigir desplazamientos largos.	Gestión manual Los centros necesitan controlar reservas, devoluciones, incidencias y permisos.
Catálogo único Búsqueda por categoría, disponibilidad y centro de recogida.	Reserva guiada Fianza transparente y seguimiento desde la cuenta personal.	Panel operativo Aprobación de solicitudes, inventario y administración de roles.

Recorrido principal

1 Explora	2 Reserva	3 Recoge	4 Devuelve
--------------	--------------	-------------	---------------

02 · ARQUITECTURA

Separación clara, flujo predecible.

Monorepo dividido por responsabilidades para que cualquier persona pueda localizar, entender y extender el código sin conocer previamente el proyecto.

FRONTEND · React + Next.js Rutas y layouts • Hooks y contexto • TanStack Query • Mapa con Leaflet	BACKEND · Express + Node Controladores • Servicios • Middlewares • Validadores	DATOS · MongoDB Atlas Modelos Mongoose • Transacciones • Índices únicos • Coordenadas GeoJSON
---	--	---

Servicios transversales

Autenticación JWT + bcrypt; permisos por rol y propietario.	Imágenes Multipart en memoria, Cloudinary y limpieza al eliminar.
Validación Zod en la entrada y respuestas de error uniformes.	Observabilidad Health, readiness, logs y documentación OpenAPI.

Estructura

1 apps/web	2 apps/api	3 data/csv	4 docs
----------------------------------	----------------------------------	----------------------------------	------------------------------

03 · MONGODB

Cuatro colecciones, relaciones reales.

Usuarios, centros, objetos y reservas comparten referencias explícitas. Los códigos externos hacen trazable cada documento desde el Excel hasta MongoDB.

USERS role • favoriteItems[] • preferredHub	HUBS managers[] • coordinates • active	ITEMS hub • category • status	RESERVATIONS user • item • hub
---	--	---	--

Relaciones

1 preferredHub / managers	2 favoriteItems › item	3 item › hub	4 reserva › user / item / hub
---	--	------------------------------------	---

Invariantes de dominio

- Un usuario nuevo siempre entra como member; nunca elige su propio rol.
- Los favoritos no se duplican y las nuevas referencias no pisan las existentes.
- Una reserva no puede superar maxLoanDays ni solaparse con otra bloqueante.
- Cada reserva conserva el centro de recogida y la relación con usuario y objeto.
- Los objetos retirados no vuelven al catálogo público mediante una edición ordinaria.

04 · DATOS REPRODUCIBLES

512 filas listas para sembrar.

El Excel es parte del producto: está validado, documentado y exportado a CSV para que Node.js lo lea con fs antes de publicar los documentos relacionados.

60	12	180	260
usuarios	centros	objetos	reservas

Pipeline de semilla

1	2	3	4	5
fs.readFile	CSV + Zod	FK y rangos	Transacción	Resumen

Controles antes de escribir

Libro Excel Códigos y correos únicos. • Cero referencias huérfanas. • Fechas ordenadas y duración válida. • Sin solapamientos bloqueantes. • Usuarios activos en reservas futuras.	Resumen del libro KPIs y gráfico por categoría. • Usuarios: 60 registros. • Centros: 12 registros. • Objetos: 180 registros. • Reservas: 260 registros.	Resultado verificable 0 relaciones huérfanas. • 0 rangos de fecha inválidos. • 4 colecciones relacionadas. • 260/260 reservas con usuario, objeto y centro válidos.
---	--	--

05 · AUTENTICACIÓN Y ROLES

Cada acción tiene un responsable.

El middleware combina autenticación JWT, rol y propiedad del recurso. La interfaz oculta lo que no corresponde, pero la autorización decisiva siempre se aplica en el servidor.

Acción	Público	Member	Manager	Admin
Ver catálogo y centros	✓	✓	✓	✓
Crear reserva	—	✓	✓	✓
Cancelar reserva propia	—	✓	✓	✓
Aprobar / entregar	—	—	✓	✓
Gestionar inventario	—	—	✓	✓
Cambiar roles	—	—	—	✓
Eliminar otra cuenta	—	—	—	✓

Reglas delicadas cubiertas

Alta segura El cuerpo de registro nunca puede convertir al usuario en manager o admin.	Cuenta propia Un member solo elimina su cuenta y se limpian avatar, favoritos y reservas pendientes.
Último admin La aplicación evita perder el único administrador activo del sistema.	Ámbito de centro Un manager opera únicamente sobre los centros que tiene asignados.

06 · REACT Y UX

Una interfaz que explica antes de exigir.

El diseño reduce incertidumbre: muestra disponibilidad, centro, duración, fianza y siguiente paso antes de iniciar una reserva.

1 /	2 /explorar	3 /objetos/:slug	4 /centros	5 /acceso	6 /cuenta	7 /gestion
--------	----------------	---------------------	---------------	--------------	--------------	---------------

Hooks con una finalidad concreta

useReducer Filtros del catálogo como estado explícito y fácil de extender.	useDeferredValue La escritura permanece fluida mientras cambia la búsqueda.	useTransition Las categorías se actualizan sin bloquear la interacción.
useOptimistic La cuenta responde al instante al cancelar una reserva.	Context + reducer Sesión, rol y modo demo coherentes en toda la aplicación.	TanStack Query Caché, estados de carga, error y reintento para datos remotos.

Mapa interactivo de centros

Leaflet (librería adicional) Mapa real de Madrid con las coordenadas GeoJSON de cada centro; librería no vista en el curso.	Sincronización con el listado Al elegir un centro el mapa vuela hasta él y abre su ficha; al pulsar un pin se selecciona en el listado.	Detalle de UX Teselas claras u oscuras según el tema, y zoom con rueda solo al interactuar para no bloquear el scroll.
---	---	--

Sistema visual

#17372E	#DDEC94	#D17A52	#F4F1E9
Bosque	Lima	Terracota	Crema

Variables CSS para color, tipografía, spacing, radios y sombras; foco visible, skip link, navegación por teclado, aria-live y preferencias de movimiento reducido.

07 · API Y CALIDAD

Seguridad y mantenibilidad medibles.

La API prioriza contratos consistentes, validación temprana y operaciones atómicas en los puntos donde dos peticiones podrían competir por el mismo recurso.

JWT + bcrypt Autenticación y contraseñas cifradas.	Zod Esquemas para params, query y body.	Rate limit Protección en rutas sensibles.
Helmet + CORS Cabeceras y orígenes controlados.	Transacciones Reservas y cambios críticos coordinados.	Cloudinary Subida y eliminación del fichero remoto.

Pruebas y verificación

24 / 24	0	0
pruebas del backend	errores de lint web	vulnerabilidades prod.

Casos cubiertos

1 Registro y login	2 Permisos por rol	3 Favoritos únicos	4 Reservas sin solape
1 Transiciones de estado	2 Límites de fecha	3 Errores uniformes	4 Health y readiness

08 · DESPLIEGUE EN PRODUCCIÓN

Frontend, API y base de datos publicados.

La aplicación completa está desplegada y es accesible públicamente: el frontend y la API en Render, y la base de datos en MongoDB Atlas. El repositorio incluye el `render.yaml` con la infraestructura como código.

Pieza	URL pública
Aplicación web	https://renodo-web.onrender.com
API REST	https://renodo-api.onrender.com/api/v1
Documentación Swagger	https://renodo-api.onrender.com/api-docs
Repositorio	https://github.com/xtragerdev/-RTC-Proyecto-Final
Base de datos	MongoDB Atlas (512 documentos sembrados)

Nota sobre el plan gratuito

Frontend y API están en el plan gratuito de Render, que suspende los servicios tras 15 minutos sin tráfico (*cold start*). Si la primera carga tarda alrededor de un minuto, es el comportamiento esperado: la primera petición reactiva los servicios y, a partir de ahí, todo responde con normalidad.

Cómo revisarlo en 5 minutos

1. Inicio y Explorar: el catálogo se carga en vivo desde la API (180 objetos).
2. Ficha de objeto: condición, fianza reembolsable, impacto y centro de recogida.
3. Reserva: «Solicitar reserva» registra la solicitud y muestra su estado.
4. Roles en /acceso: accesos demostrativos de Miembro, Responsable y Admin.
5. Centros: mapa interactivo real con Leaflet sincronizado con el listado.
6. API: documentación interactiva en Swagger (/api-docs).

Acceso de prueba

Cuenta real sembrada: `iker-navarro.0002@renodo.example` con contraseña **ReNodoDemo2026!** — todas las cuentas del dataset son sintéticas. En /acceso también hay tres accesos demostrativos locales.

09 · ENTREGA FINAL

Preparado para corregir, ejecutar y ampliar.

El repositorio reúne aplicación, datos, documentación, capturas del despliegue, ejemplos de entorno y configuración de infraestructura. Cada requisito puede verificarse desde una ruta concreta.

- ✓ Variables CSS y estilos reutilizables
- ✓ Tres colecciones relacionadas además de usuarios
- ✓ Excel con más de 100 filas y semilla mediante fs
- ✓ Roles, autenticación y autorización por ámbito
- ✓ Cloudinary desde formulario multipart
- ✓ Hooks avanzados con propósito visible
- ✓ Mapa interactivo con Leaflet y capturas reales en docs/capturas
- ✓ Frontend y backend desplegados en Render con MongoDB Atlas

Enlaces de entrega

Aplicación https://renodo-web.onrender.com	API + Swagger https://renodo-api.onrender.com/api/v1 · /api-docs
Repositorio https://github.com/xtragerdev/-RTC-Proyecto-Final	Dataset data/ReNodo-dataset.xlsx + data/csv/*.csv

Conclusión

ReNodo no es una colección de pantallas aisladas: es un servicio completo con público definido, reglas de negocio, datos trazables, despliegue real y una experiencia coherente de extremo a extremo.